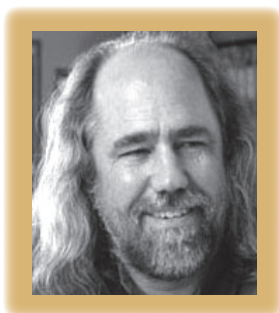


An Architectural Oxymoron

Grady Booch

Bjarne Stroustrup once suggested to me that if you can't say what a thing is not, then you haven't really explained what that thing is. So, to be clear, an oxymoron is not a bovine of meager intelligence, nor is it a chemical compound with two covalently bound oxygen atoms. Rather, an oxymoron is a figure of speech that combines two seemingly contradictory terms and unites them in an apparent paradox. Military intelligence. Civil war. Press release. Jumbo shrimp. Pretty ugly. Agile architecture.



A Really Good Idea

Now, you might think that phrase is nothing more than a simple alliteration. Others might view it as more of a euphemism. I prefer to think of it as A Really Good Idea, viewed from two perspectives.

First, consider an architecture-as-an-artifact, and then prefix “agile” as its qualifier. The architecture of all well-structured software-intensive systems should be resilient and loosely coupled; all such good architectures are grounded with a small set of well-reasoned and well-syndicated design decisions. Nonetheless, these decisions offer some wiggle room that lets the moving parts of that system adapt and adjust without bashing into one another. All such good architectures can be nudged in ways far beyond their initial expectations without major drama and without bringing its load-bearing walls down owing to the presence of brittle, unyielding structures. In short, all good software-intensive architectures are agile.

Second, think of agile as the nominative whose action is then “to architect.” Ultimately, the most important artifact of any development process is the executable system itself, the raw running

naked code that makes the system manifest. To get there, to build the right executables at the right time with the right functionality, requires an effective agile process that grows a system's architecture incrementally and iteratively along the way. One should architect in an agile manner, and the form and representation of a system's architecture are the fruits in the harvest of an agile process.

My Heroes

Scott Ambler is one of my heroes in the agile architecture space, and he's written extensively about this subject (for example, see www.youtube.com/watch?v=_SrSyXTXCok&feature=related and www.agilemodeling.com/essays/agileArchitecture.htm). Scott often speaks of agility at scale in a way that makes a great deal of sense to me: most economically interesting software-intensive systems require teams of a modest to large size, and that demands a balance between agility and predictability/repeatability. As Scott notes, the key to making agility at scale work is recognizing that there's a recursive nature to systems architecture: one man's system is another woman's subsystem, so there can be a reasonable separation of concerns in the design of each nearly independent part. Furthermore, as Scott suggests, you can grow a system's architecture best by driving the development process forward feature by feature and, I would add, paying attention to attacking risks and doing intentional, continuous refactoring along the way. As Scott points out, architecting is not an instance of Big Design Up Front (BDUF; <http://c2.com/cgi/wiki?BigDesignUpFront>). Rather, it's doing just enough design to advance development—not so much that you prematurely and unnecessarily bind decisions, yet not so little that you avoid tackling the big problems. Indeed, the SimplestThing

Continued on p. 95

Continued from p. 96

ThatCouldPossiblyWork (<http://c2.com/cgi/wiki?DoTheSimplestThingThatCouldPossiblyWork>) is one of the fundamental patterns of Extreme Programming, which helped form the roots of contemporary agile practices. Scott goes on to note that modeling has a real place in agile architecting. You do just enough to establish a common vision and a common understanding, and you document just enough to preserve the most important common memes for future stakeholders, focusing primarily on design decisions that are not obvious from the code itself.

Another one of my heroes is Jim Coplien who—along with Gertrud Bjornvig—has just published a new book on the subject, *Lean Architecture* (John Wiley & Sons, 2010). They offer these observations about architecture:

- Architecture is more about form than structure.
- Architecture is more about compression than abstraction.
- Much architecture is not about solving users' problems.
- Architecture has both a static as well as a dynamic dimension.
- Architecture is everybody's job.
- Architecture need not be hard.

There's much to be discussed about all of these insights, but I want to focus on three that particularly resonate with my experience.

More about Form than Structure

To say that architecture is more about form than structure may at first seem paradoxical. This is not to say that structure is irrelevant; we still need to establish the load-bearing walls, the largely immutable architectural decisions that establish the system's skeleton to which we incrementally add the meat of the implementation. However, as the authors note, it's more important to establish the style, the design patterns if you will, because these are the guiding principles that shape the implementation. As an example, I recall an engagement at a large financial institution where, in an archeological dig, we uncovered the decision that all state-changing operations would be encapsulated in a stored

procedure. At first, I thought this was just an incidental programming idiom, resulting from some historical precedent. However, with further investigation, it became clear that this was an important style: by ensuring that all such state-changing operations were consigned to the database, it meant that the opportunities for malicious coding were far less. Imagine shaving a few cents off random transactions; after a few million such transactions spread over time, you'd end up with some Real Money and no one would have noticed, as each small theft would have been below the radar of concern. However, by constraining state-changing events to stored procedures, this isolated such code to parts of the system where there were strong, formal procedures for change management. Thus, we have a style that impacted both programmatic as well as social structure.

More than Just Attending to Users' Problems

I especially appreciate Jim and Gertrud's observation that architecture is about more than just attending to the user's problems. The reality of any economically interesting software-intensive system is that the role of a "user" is quite broad. There will be many different stakeholders with many concepts and particular problems, and optimizing for one set of stakeholders generally requires compromises for others. Furthermore, what a user is today is not what a user may be tomorrow, for the very presence of a system changes the perceptions and needs of today's users, demanding that the system be resilient to change. Thus, as the authors note, to properly architect a system, you have to care about form, not just function. In this, there's some apparent tension between what Scott suggests (with feature-driven releases) and what Jim and Gertrud observe (to focus on form). Actually, I see only a healthy tension in this duality, so my recommendation is to seek moderation in all things (and, as is often said, one must also seek moderation in moderation).

Both Static and Dynamic Components

I also resonate with the observations in *Lean Architecture* about architecture encompassing both static and dynamic elements. Too often, I see architects care only about structure, and in so doing neglect dynamics. Indeed, this is one of the reasons

I'm such a fan of Philippe Kruchten's 4+1 model view (www.computer.org/portal/web/csdl/doi/10.1109/52.469759); in it he emphasizes the presence of both the static and the dynamic. His process view is often misunderstood in that regard, but this is the point of view where one attends to concurrency, performance, scalability, and throughput. In many classes of systems, developers may assume a single thread of control, but except for the simplest systems, this is just not the case. Many processes and threads will often be running at once, some on the same node, others not, and failing to be intentional about these structures will make it very difficult to grow the system later. Especially in the case of embedded systems, the choice of a time-triggered pattern versus an event-driven one is one of the most important dynamic architectural decisions to be made, perhaps even more important than many other structural decisions.

Antoine de Saint-Exupéry once said, "A designer knows he has achieved perfection not when there is nothing left to add, but when there is nothing left to take away." To reframe it in our domain, given a system, remove everything that is not fundamental. What remains is its essential architecture.

In an agile context, how does that essence manifest itself? At best, it should be manifest in the code itself, visible and approachable and self-evident. As the complexity of the system grows, you must make that architecture manifest in ways beyond the code. At the very least, track the 10 most important design decisions you've made, and defend their use ruthlessly (but challenge their assumptions and refactor them as needed). If you need yet more governance, make visible the cross-cutting design patterns that give the system form. If you need even more, document the system from the perspective of its essential views. And, in all cases, socialize the architecture so that it becomes part of the tribal memory. 🍷

Grady Booch is an IBM Fellow and one of the UML's original authors. He also developed the Booch method of software development (*Object-Oriented Analysis and Design*, Addison-Wesley, 1993). He's working on a handbook of architectural patterns, available at www.handbookofsoftwarearchitecture.com. Contact him at architecture@booch.com.